UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

AImage Lab

Laurea Magistrale in Ingegneria Informatica
a.a. 2023-2024

Scalable AI

State Space Models, Introduction to Mamba

Introduction

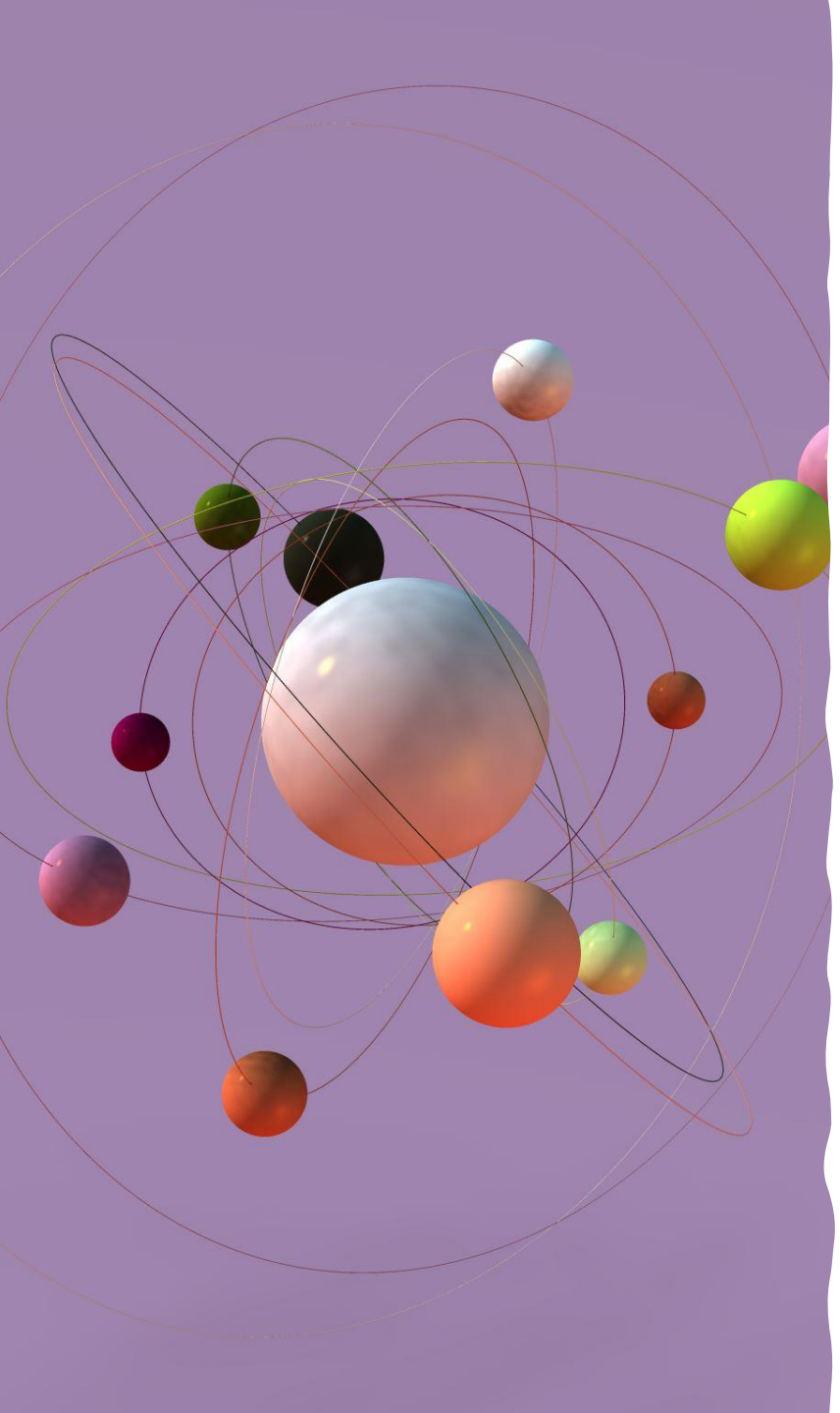
- Foundation models (FMs), or large models pretrained on massive data then adapted for downstream tasks
- Modern FMs are predominantly based on a single type of sequence model: the Transformer and its core attention layer
- Problem with Transformers: an inability to model anything outside of a finite window, and quadratic scaling with respect to the window length.
- Structured state space sequence models (S4) intend to address these issues

Structured State Space for Sequence Modeling (S4)

- Many subquadratic-time architectures such as linear attention, gated convolution and recurrent models, and structured state space models (SSMs) have been developed to address Transformers' computational inefficiency on long sequences
- SSM models can be interpreted as a combination of recurrent neural networks (RNNs) and convolutional neural networks (CNN)

Structured State Space for Sequence Modeling (S4)

- a new approach to very long-range sequence modeling tasks for vision, language, and audio
- Have a capacity to capture dependencies over tens of thousands of steps.



State Space Models

- **Goal:** efficient modeling of long sequences
- **Solution:** build a new neural network layer based on State Space Models.
- RNN can be used but they are sequentially computed.
- Transformers are not sequential like RNN but they come with heavy computation cost with longer sequence
- Can we do something better? *Maybe with "state space model"*

But what is state space model?

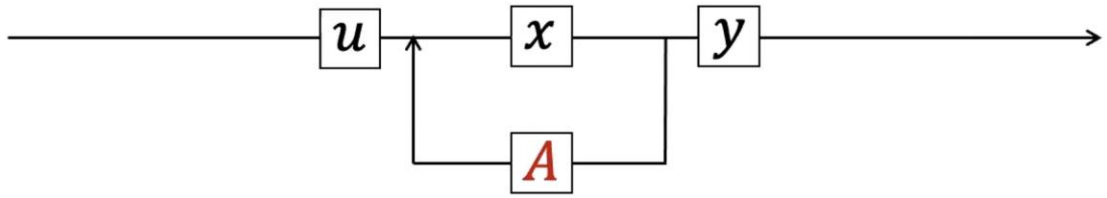
What is state space model?

- It maps a 1-Dimensional input signal $u(t)$ to an N-Dimensional latent state $\mathrm{x}(t)$
- Then it projects the result to a 1-Dimensional output signal $\mathrm{y}(t)$
- Note: The input and output can be multi-dimensional, for simplicity we assume here it is 1-dimensional
- SSM is used as a black-box representation in a deep sequence model**, where $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}, \boldsymbol{D}$ are parameters learned by gradient descent.**
- Note: We discard the term D for simplicity in next follow-up equations

$$x'(t) = \boldsymbol{A}x(t) + \boldsymbol{B}u(t)$$
$$y(t) = \boldsymbol{C}x(t) + \boldsymbol{D}u(t)$$

What is state space model?

- u is the input function
- x is the latent function
- y is the output function
- SSM first introduced in control models in 1960s
- x usually has higher dimension than u and y
- u, y, x are functions of time

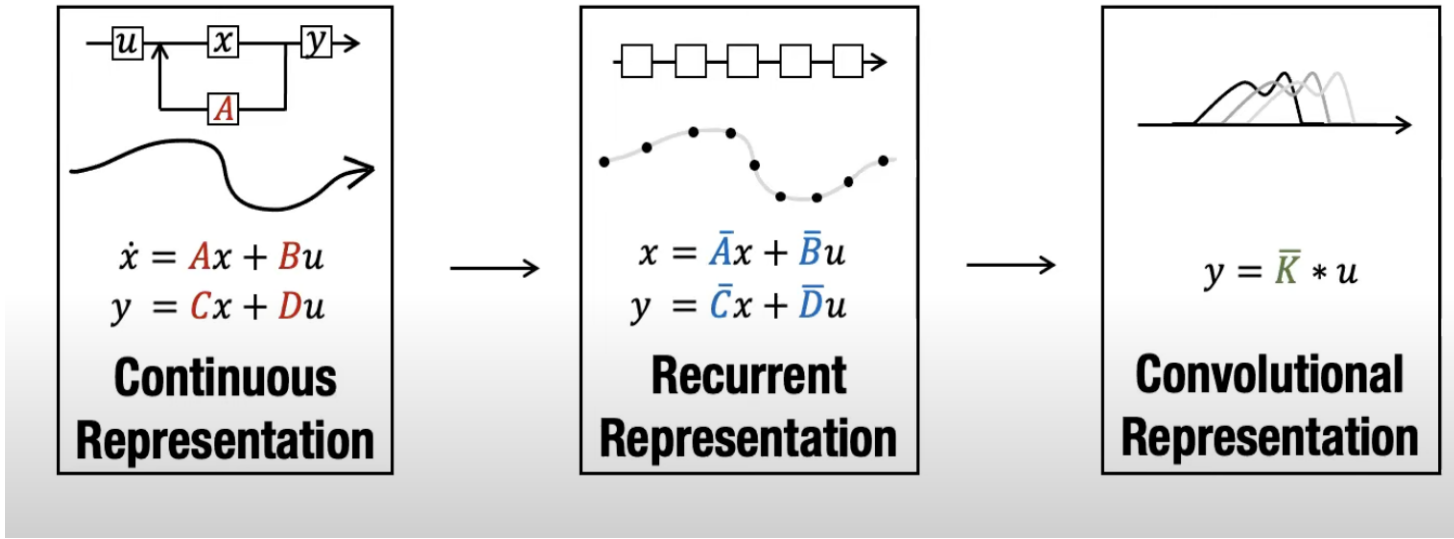


$$x' = Ax + Bu$$
$$y = Cx + Du$$

R. E. Kalman. "A New Approach to Linear Filtering and Prediction Problems." ASME 1960

Properties of SSM

- It is a sequence-to-sequence map
- You can use SSM for:
  - 1- Continuous models
  - 2- Recurrent models
  - 3- Convolutional models



Source: https://srush.github.io/annotated-s4/

Discrete-time SSM: The Recurrent Representation

- What to do if instead of having continuous function $u(t)$, we have discrete inputs sequence like:
  - $u_0, u_1, \dots,$
- SSM must be discretized by a step size Δ
- Δ represents the resolution of the input.
- u_i can be viewed as a sample from function $u(t)$
- To discretize the continuous-time SSM, we can use the bilinear method
- It converts $\boldsymbol{A}$ to approximation of $\overline{\boldsymbol{A}}$

Continuous-time system equations:

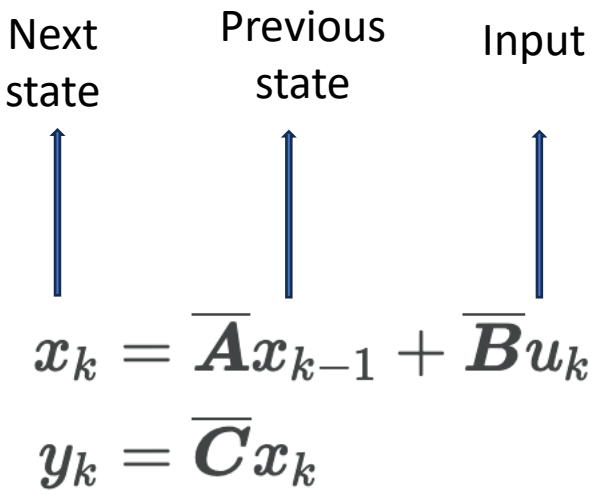
- $x'(t) = Ax(t) + Bu(t)$
- $y(t) = Cx(t) + Du(t)$

Euler Discretization:

- $x(t + \Delta) \approx x(t) + \Delta x'(t)$
- $= x(t) + \Delta(Ax(t) + Bu(t))$
- $= (I + \Delta A)x(t) + \Delta Bu(t)$
- $= \bar{A}x(t) + \bar{B}u(t)$

Discrete-time SSM: The Recurrent Representation

- How to change the transformation equation for $\mathrm{u}(t) \rightarrow y(t)$ given discrete input sequence?
- Instead of working with $\mathrm{u}(t) \rightarrow y(t)$, we work with $\mathrm{u_k} \rightarrow y_k$ to indicate time-steps
- Now the equation is recurrence on x_k
- We can use RNN model to compute state of SSM
- x_k is basically the hidden state carried out in RNN
- RNN are more efficient at inference time due to constant state size independent of sequence length (faster at testing)
- But they cannot get parallelized like transformers (problem during training)



Discrete-time SSM: A Mechanics Example

- Consider a mass attached to a wall by spring with forward position $p(t)$
- Varying force $\mathrm{u}(t)$ is applied to the mass
- Parameters of the system:
  - mass: m
  - Spring constant: k
  - Friction constant: b
- We can relate everything with the following Differential Equation:

$$mp''(t) = u(t) - bp'(t) - kp(t)$$

Discrete-time SSM: A Mechanics Example

- We can rewrite this equation with $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}$ of SSM:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ -k/m & -b/m \end{bmatrix}$$

$$\boldsymbol{B} = \begin{bmatrix} 0 \\ 1/m \end{bmatrix} \qquad \boldsymbol{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

- $\boldsymbol{C}$ shows the first dimension of hidden state is the position
- Because it becomes $y(t)$
- The second dimension is velocity as it is impacted by $u(t)$ through B
- A relates these terms

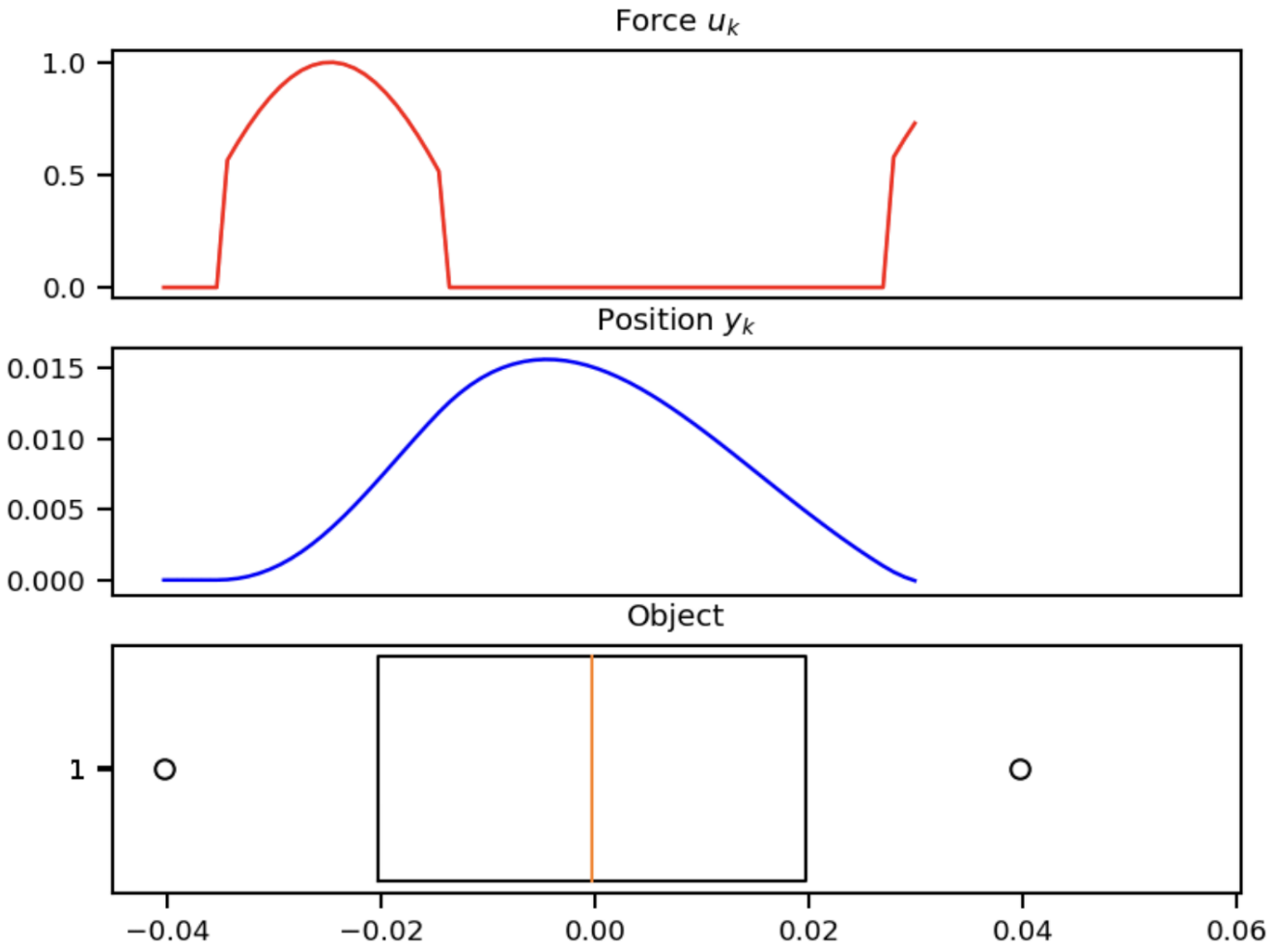
Discrete-time SSM: A Mechanics Example

- $\boldsymbol{C}$ shows the first dimension of hidden state is the position
- Because it becomes $y(t)$
- The second dimension is velocity as it is impacted by $u(t)$ through B
- A relates these terms
- The following code put everything together to simulate SSM modeling of the problem

```
def example_ssm():
    # SSM
    ssm = example_mass(k=40, b=5, m=1)

    # L samples of u(t).
    L = 100
    step = 1.0 / L
    ks = np.arange(L)
    u = example_force(ks * step)

    # Approximation of y(t).
    y = run_SSM(*ssm, u)

    # Plotting ---
    import matplotlib.pyplot as plt
    import seaborn
    from celluloid import Camera

    seaborn.set_context("paper")
    fig, (ax1, ax2, ax3) = plt.subplots(3)
    camera = Camera(fig)
    ax1.set_title("Force $u_k$")
    ax2.set_title("Position $y_k$")
    ax3.set_title("Object")
    ax1.set_xticks([], [])
    ax2.set_xticks([], [])
```

Discrete-time SSM: A Mechanics Example

- Position is predicted by SSM matches with object position
- The force is asserted to the object to change the position over time
- This example is just 1 SSM, with 2 hidden states over 100 steps
- We can stack many SSM blocks in our network



Training SSMs: The Convolutional Representation

- we can turn the “RNN” representation into a “CNN” by unrolling it
- Why want CNN representation? Training RNN is sequential
- **Recurrent SSM can be written as a discrete convolution**
- Let’s see the equations by unrolling them

$$x_k = \overline{\boldsymbol{A}} x_{k-1} + \overline{\boldsymbol{B}} u_k \qquad x_0 = \overline{\boldsymbol{B}} u_0 \qquad x_1 = \overline{\boldsymbol{AB}} u_0 + \overline{\boldsymbol{B}} u_1 \qquad x_2 = \overline{\boldsymbol{A}}^2 \overline{\boldsymbol{B}} u_0 + \overline{\boldsymbol{AB}} u_1 + \overline{\boldsymbol{B}} u_2 \qquad \dots$$

$$y_k = \overline{\boldsymbol{C}} x_k \qquad y_0 = \overline{\boldsymbol{CB}} u_0 \qquad y_1 = \overline{\boldsymbol{CAB}} u_0 + \overline{\boldsymbol{CB}} u_1 \qquad y_2 = \overline{\boldsymbol{CA}}^2 \overline{\boldsymbol{B}} u_0 + \overline{\boldsymbol{CAB}} u_1 + \overline{\boldsymbol{CB}} u_2 \qquad \dots$$

Training SSMs: The Convolutional Representation

- We can vectorize this as follows and get the closed formula:

$$y_k = \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}^k\overline{\boldsymbol{B}}u_0 + \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}^{k-1}\overline{\boldsymbol{B}}u_1 + \cdots + \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}\overline{\boldsymbol{B}}u_{k-1} + \overline{\boldsymbol{C}}\overline{\boldsymbol{B}}u_k$$

$$y = \overline{\boldsymbol{K}} * u$$

$$\overline{\boldsymbol{K}} \in \mathbb{R}^L = (\overline{\boldsymbol{C}}\overline{\boldsymbol{B}}, \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}\overline{\boldsymbol{B}}, \ldots, \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}^{L-1}\overline{\boldsymbol{B}})$$

- $\overline{K}$ is called the **SSM convolution kernel or filter**
- $\overline{K}$ is a giant filter applied to input u

Now output can be computed without computing the internal state u(t) -> making it parallel

Training SSMs: The Convolutional Representation

- Is the problem solved? No!
- $\overline{K}$ (the **SSM convolution kernel**) is just too big!
- Solution? Use Fast Fourier Transform (FFT) using convolution theorem
- How?
  - The discrete convolution theorem allows us to efficiently calculate the output of convolution by first multiplying FFTs of the input sequences
  - Next applying an inverse FFT
- So, instead of sweeping the whole input sequence with a large filter which takes O(NM), where N is the sequence length and M is the filter length
- For FFT it is O$((N + M)\log(N + M))$

The Convolution Theorem

Convolution Theorem

The Fourier transform of a convolution of two signals is the product of their Fourier transforms: $f \circledast g \leftrightarrow FG$. The convolution of two continuous signals f and g is

$$(f \circledast g)(x) = \int_{-\infty}^{+\infty} f(t)g(x-t)\,dt$$

So $\int_{-\infty}^{+\infty} f(t)g(x-t)\,dt \leftrightarrow F(\omega)G(\omega)$.

The Fourier transform of a product of two signals is the convolution of their Fourier transforms: $fg \leftrightarrow F \circledast G/2\pi$.

Training SSMs: FFT Transform code

```
def causal_convolution(u, K, nofft=False):
    if nofft:
        return convolve(u, K, mode="full")[: u.shape[0]]
    else:
        assert K.shape[0] == u.shape[0]
        ud = np.fft.rfft(np.pad(u, (0, K.shape[0])))
        Kd = np.fft.rfft(np.pad(K, (0, u.shape[0])))
        out = ud * Kd
        return np.fft.irfft(out)[: u.shape[0]]
```

- **nofft**: a boolean flag to choose between direct convolution (True) and FFT-based convolution
- **np.fft.rfft**: This function computes the one-dimensional n-point discrete Fourier Transform (DFT) of a real-valued array by means of an efficient algorithm called the Fast Fourier Transform (FFT)
- **np.fft.irfft:** is the inverse FFT function to convert it back to the time domain

S4 & HiPPO Matrix: Addressing Long range dependency

- It has been shown that basic SSM performs very poorly in practice
- Why? vanishing/exploding gradients problem
- They suffer from gradients scaling exponentially in the sequence length
- For long-term dependency, we need something better
- Use HiPPO Matrix in SSM to get S4
- S4 is a special SSM

What is S4

SSM + HiPPO + Structured Matrices = S4

- HiPPO: Use special formulas for A, B Matrices in SSM

$$x'(t) = Ax(t) + Bu(t)$$
$$y(t) = Cx(t) + Du(t)$$

- Matrix A is basically can be seen as the matrix that captures the previous state. It also decides how the information is copied.

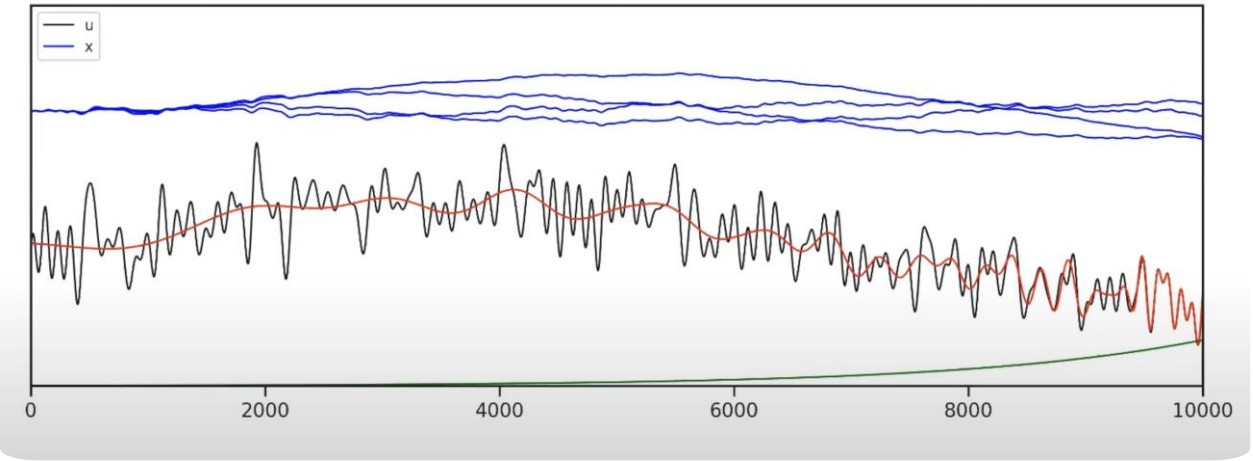
HiPPO Matrix:

$$x'(t) = \boldsymbol{A}x(t) + \boldsymbol{B}u(t)$$ HiPPO operator

$$(\textbf{HiPPO Matrix}) \qquad \boldsymbol{A}_{nk} = \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}$$

- HiPPO specifies a class of certain matrices $A \in R^{n \times n}$
- **Motivation**: The matrix allow the state $x(t)$ to memorize the history of the input $u(t)$
- It is shown that simply modifying an SSM from a random matrix A to HiPPO improved its performance on the sequential MNIST classification benchmark from 60% to 98%
- HiPPO matrix aims to compress the past history into a state (x) that has enough information to approximately reconstruct the history

Example of HiPPO: Reconstruction of function

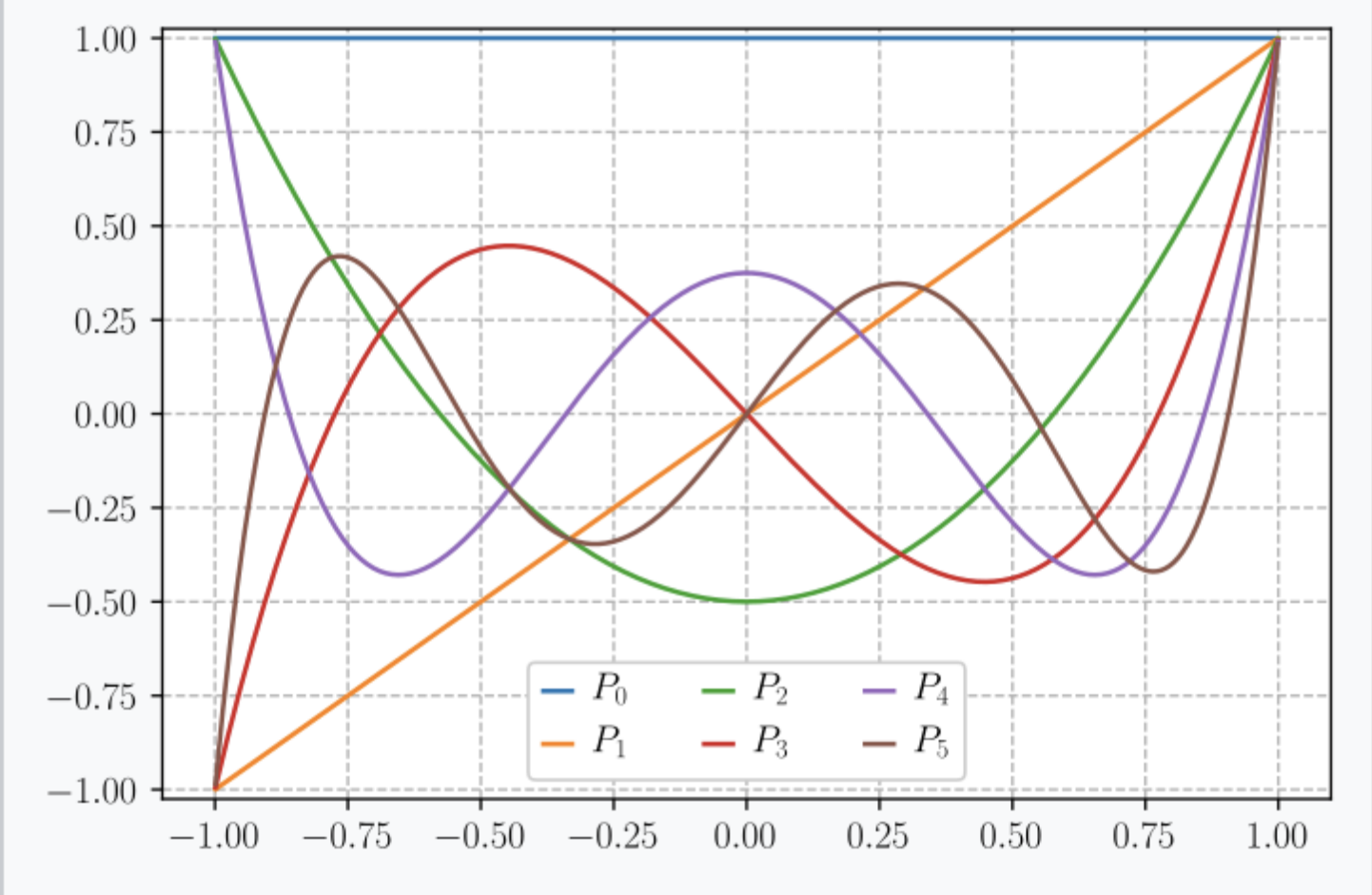


credit: https://srush.github.io/annotated-s4/

- u(t) is our input function
- We pass it through HiPPO to get the higher dimension latent state x(t)
- x(t) is a vector of dimension size 64
- We use linear projection to get red line for reconstruction function y(t)
- After 10000 steps it can perfectly reconstruct the most recent part of the function
- As we go backward in time, the accuracy of reconstruction drops as our state size 64 << 10000 steps
- Green line is the weight matrix for reconstruction, saying that recent history matters more than older one
- It can be used for online function reconstruction

How HiPPO Matrix keep track of history?

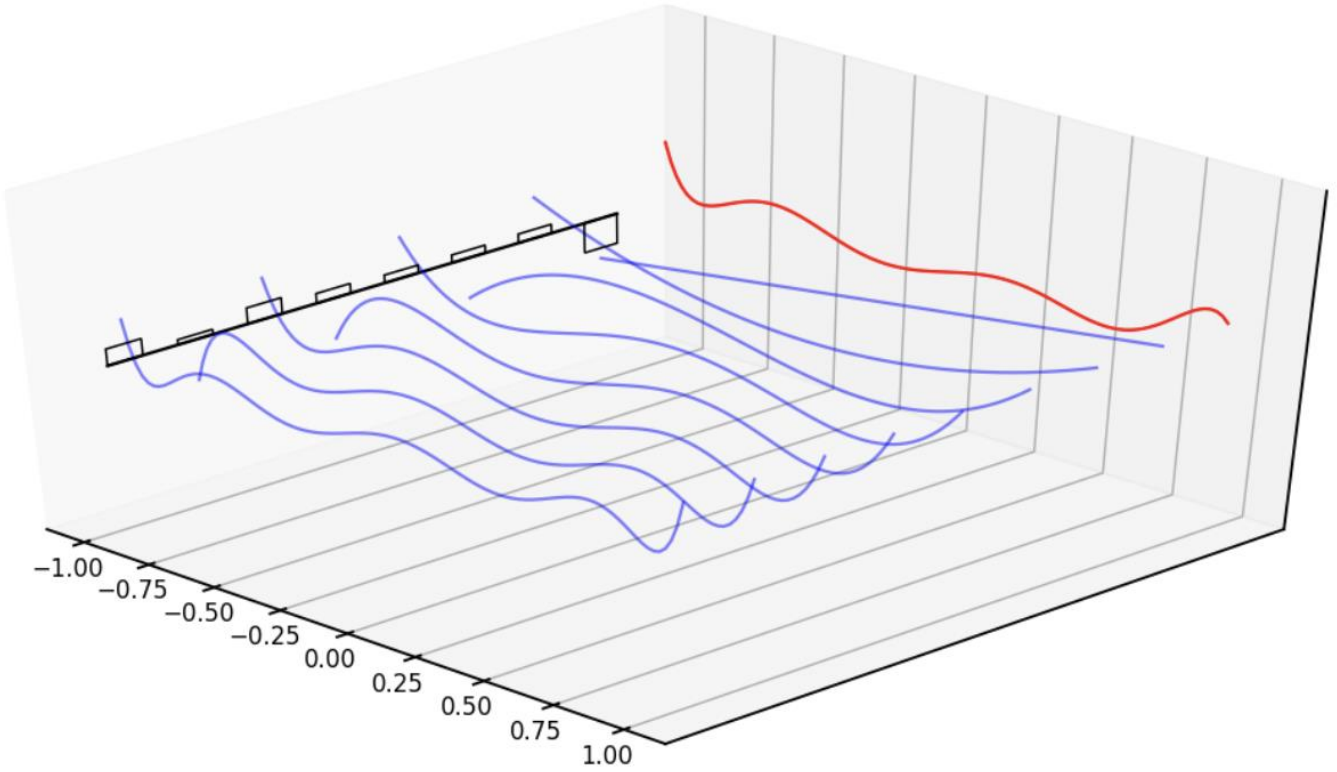
- It produces a hidden state that memorizes its history
- keeps track of the coefficients of a Legendre polynomial
- These coefficients let it approximate all of the previous history
- Legendre polynomial are a system of complete and orthogonal polynomials



The first six Legendre polynomials

How HiPPO Matrix keep track of history? Example

- Red line represents that curve we are approximating
- Black bars represent the values of our hidden state
- Each is a coefficient for one element of the Legendre series shown as blue functions
- HiPPO matrix updates these coefficients each step



SSM Neural Network

- We now have all of the ingredients needed to build a basic SSM neural network layer
- We assume that we are going to be learning the parameters:
  - B
  - C
  - step size Δ
  - scalar D
- The code on the right shows torch implementation of SSM Layer

```
class SSMLayer(nn.Module):
    N: int
    l_max: int
    decode: bool = False

    def setup(self):
        # SSM parameters
        self.A = self.param("A", lecun_normal(), (self.N, self.N))
        self.B = self.param("B", lecun_normal(), (self.N, 1))
        self.C = self.param("C", lecun_normal(), (1, self.N))
        self.D = self.param("D", nn.initializers.ones, (1,))

        # Step parameter
        self.log_step = self.param("log_step", log_step_initializer(), (1,))

        step = np.exp(self.log_step)
        self.ssm = discretize(self.A, self.B, self.C, step=step)
        self.K = K_conv(*self.ssm, self.l_max)

        # RNN cache for long sequences
        self.x_k_1 = self.variable("cache", "cache_x_k", np.zeros,
(self.N,))

    def __call__(self, u):
        if not self.decode:
            # CNN Mode
            return causal_convolution(u, self.K) + self.D * u
        else:
            # RNN Mode
            x_k, y_s = scan_SSM(*self.ssm, u[:, np.newaxis],
self.x_k_1.value)
            if self.is_mutable_collection("cache"):
                self.x_k_1.value = x_k
            return y_s.reshape(-1).real + self.D * u
```

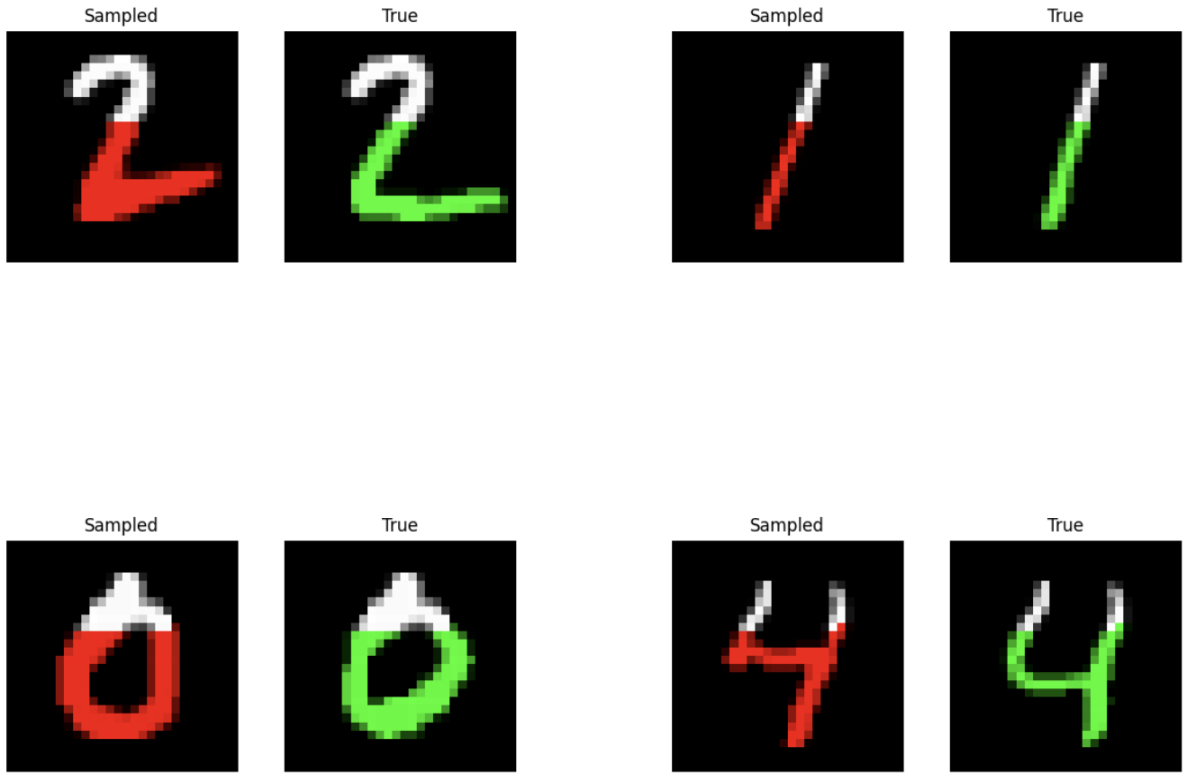
SSM Neural Network

- The SSM Layer can be put into standard NN layer
- Here the dropout and layer norm is added

```
class SequenceBlock(nn.Module):
    layer_cls: nn.Module
    layer: dict  # Hyperparameters of inner layer
    dropout: float
    d_model: int
    prenorm: bool = True
    glu: bool = True
    training: bool = True
    decode: bool = False

    def setup(self):
        self.seq = self.layer_cls(**self.layer, decode=self.decode)
        self.norm = nn.LayerNorm()
        self.out = nn.Dense(self.d_model)
        if self.glu:
            self.out2 = nn.Dense(self.d_model)
        self.drop = nn.Dropout(
            self.dropout,
            broadcast_dims=[0],
            deterministic=not self.training,
        )

    def __call__(self, x):
        skip = x
        if self.prenorm:
            x = self.norm(x)
        x = self.seq(x)
        x = self.drop(nn.gelu(x))
        if self.glu:
            x = self.out(x) * jax.nn.sigmoid(self.out2(x))
        else:
            x = self.out(x)
        x = skip + self.drop(x)
        if not self.prenorm:
            x = self.norm(x)
        return x
```

Experiment: Generating MNIST digits

- Task: Given the initial the first few pixels, generate the rest of MNIST digit
- It should predict the entire sequences of pixels
- How it works? we feed in a sequence of pixels into the model and it predict the next one
- Like language modeling

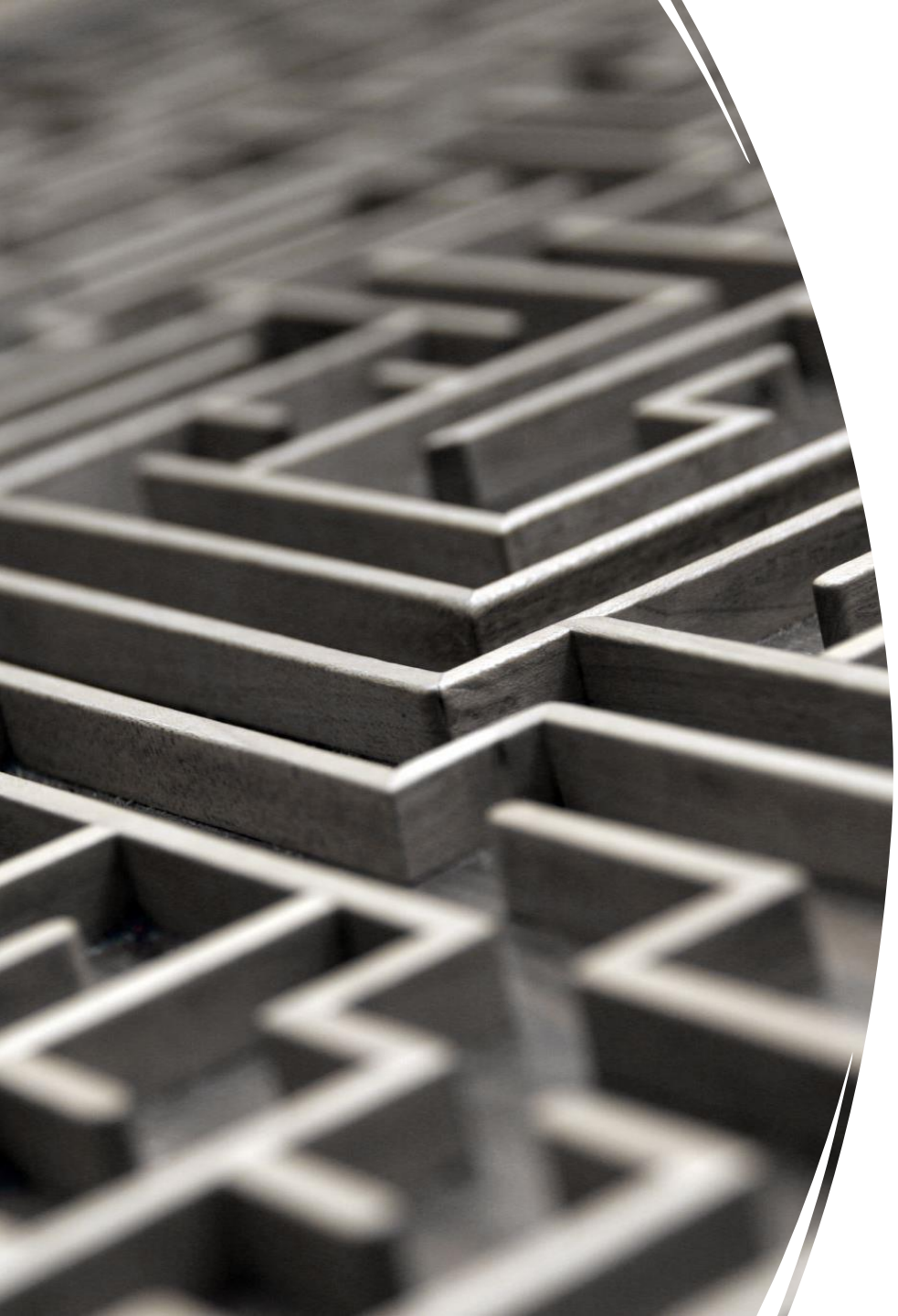


S4 is on the left and ground truth is on the right.
White pixels shows the initial state

SSM weaknesses

- They have not performed as well as attention on important modalities such as language.
- key weakness: inability to perform content-based reasoning
- Mamba tries to address these weaknesses

Mamba



Mamba

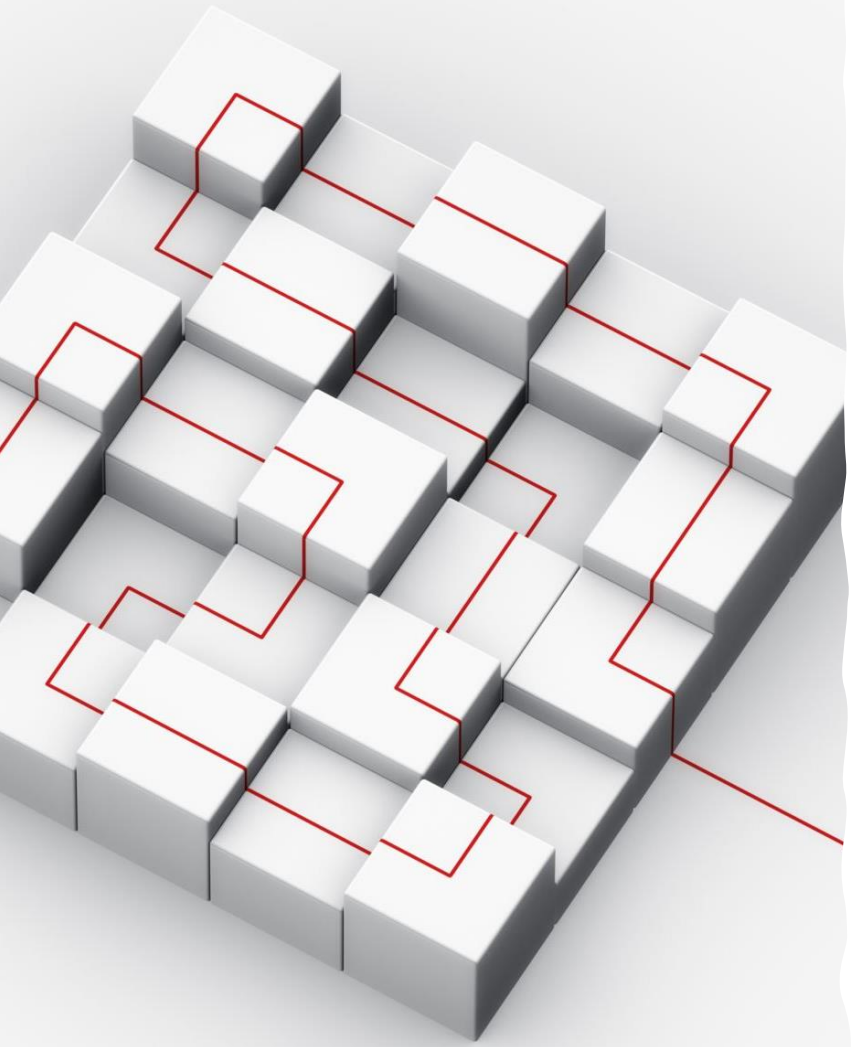
- Mamba has several improvements in comparison with previous methods:
  - Introduces discrete modalities, allowing the model to selectively propagate or forget information along the sequence length
  - Introduces hardware-aware parallel algorithm in recurrent mode
- these selective features are integrated into a simplified end-to-end neural network architecture without attention or even MLP blocks

Benefits of Mamba

- It has faster inference (5× higher throughput than Transformers)
- It has linear scaling in sequence length
- These features allows Mamba to be used on larger sequence length up to millions
- It achieves state-of-the-art performance across several modalities such as language, audio, and genomics
- It proposes new class of SSM to cover the weaknesses of preceding models

Improvement

- **Selection Mechanism**
  - the ability to efficiently select data in an input-dependent manner
  - focus on or ignore particular inputs
  - Introduces a simple selection mechanism by parameterizing the SSM parameters based on the input
  - This allows the model to filter out irrelevant information and remember relevant information indefinitely



Improvement

- **Architecture**
  - Simplify prior designs
  - MLP block of Transformers is compressed into a single block
  - leads to a simple and homogenous architecture design
  - Includes **selective state spaces**

Selective State Space block

- It is fully recurrent models with key properties
  - High quality: selectivity brings strong performance on dense modalities such as language
  - Long context: the quality and efficiency together yield performance improvements on real data up to sequence length 1M
- As it can be seen, Δ, B, C is not fixed but dependent on input x_t

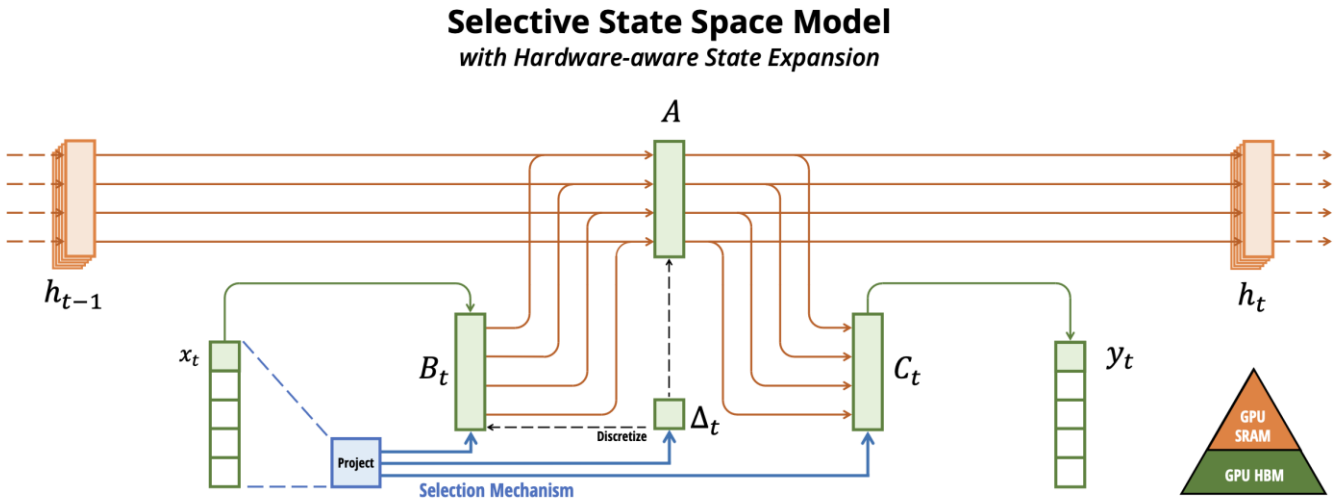


Figure 1: (**Overview**.) Structured SSMs independently map each channel (e.g. $D = 5$) of an input x to output y through a higher dimensional latent state h (e.g. $N = 4$). Prior SSMs avoid materializing this large effective state (DN, times batch size B and sequence length L) through clever alternate computation paths requiring time-invariance: the ($\Delta, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}$) parameters are constant across time. Our selection mechanism adds back input-dependent dynamics, which also requires a careful hardware-aware algorithm to only materialize the expanded states in more efficient levels of the GPU memory hierarchy.

Credit: Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752

Selective State Space (Motivation)

- Fundamental problem of sequence modeling is compressing context into a smaller state.
- Attention is both effective and inefficient
- Autoregressive inference requires explicitly storing the entire context
- On the other hand, recurrent models are efficient because they have a finite state
- Recurrent models effectiveness is limited by how well this state has compressed the context
- Efficient models must have a small state
- Effective models must have a state that contains all necessary information from the context.
- Mamba achieves this balance by **using selection**

Selective State Space Example (Motivation)

- Consider two simple tasks:
  - Copying: The model should produce the same output as input but time shifted. This can be done with S4
  - Selective copying: The model should only output colored tokens and not the white ones. Example: Removing bad words from tweet.
    Here S4 fails, mamba wins
- Incorporating a selection mechanism into models provides more efficient solution
- It keeps ignoring irrelevant inputs
- White tokens in selective copying is irrelevant
- S4 fails because it cannot do content-aware reasoning

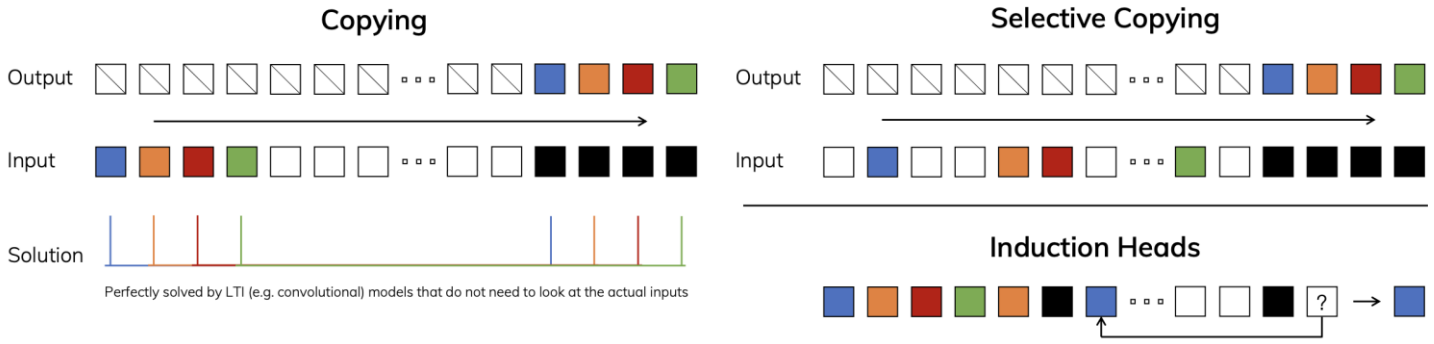


Figure 2: (*Left*) The standard version of the Copying task involves constant spacing between input and output elements and is easily solved by time-invariant models such as linear recurrences and global convolutions. (*Right Top*) The Selective Copying task has random spacing in between inputs and requires time-varying models that can *selectively* remember or ignore inputs depending on their content. (*Right Bottom*) The Induction Heads task is an example of associative recall that requires retrieving an answer based on context, a key ability for LLMs.

Credit: Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752

Selective State Space Example (Motivation)

- Consider another simple task:
  - Induction Head: The model require to recall information from previous input to build the next output. For example: remembering after each blac token, blue token comes. S4 fails, Mamba wins
- The main reason why S4 fails is because all it matrices A, B, C, D are constant over time and does not change.
- So, S4 it loses its content awareness over time.

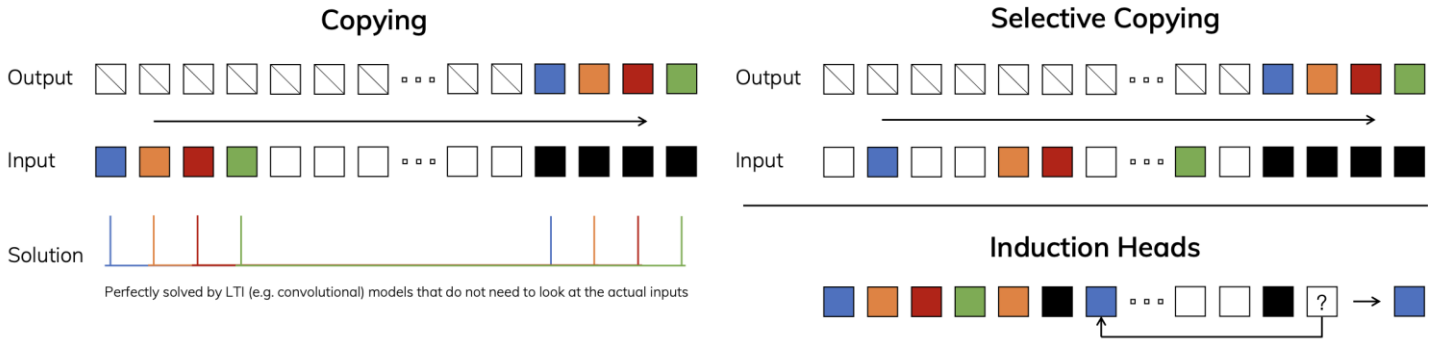


Figure 2: (*Left*) The standard version of the Copying task involves constant spacing between input and output elements and is easily solved by time-invariant models such as linear recurrences and global convolutions. (*Right Top*) The Selective Copying task has random spacing in between inputs and requires time-varying models that can *selectively* remember or ignore inputs depending on their content. (*Right Bottom*) The Induction Heads task is an example of associative recall that requires retrieving an answer based on context, a key ability for LLMs.

Credit: Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752

Main Difference between Mamba and S4

- The main difference is simply making several parameters functions of the input
  - Δ, B, C
- Algorithm S6 (Mamba) is time-varying
- Algorithm S4 is time-invariant
- **Mamba relaxes the idea of time transition between input sequences are independent**

Algorithm 1 SSM (S4)

Input: $x : (\mathtt{B}, \mathtt{L}, \mathtt{D})$ (Batch size, sequence length, token dimension)
Output: $y : (\mathtt{B}, \mathtt{L}, \mathtt{D})$

Dimension of hidden state is N, A captures history

$\boldsymbol{A} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$\boldsymbol{B} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
$\boldsymbol{C} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
$\Delta : (\mathtt{D}) \leftarrow \tau_\Delta$(Parameter) Step size for discretization (learned)
$\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathtt{D}, \mathtt{N}) \leftarrow$ discretize$(\Delta, \boldsymbol{A}, \boldsymbol{B})$
$y \leftarrow \mathrm{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
return y

Algorithm 2 SSM + Selection (S6)

Input: $x : (\mathtt{B}, \mathtt{L}, \mathtt{D})$
Output: $y : (\mathtt{B}, \mathtt{L}, \mathtt{D})$

$\boldsymbol{A} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$\boldsymbol{B} : (\mathtt{B}, \mathtt{L}, \mathtt{N}) \leftarrow s_B(x)$
$\boldsymbol{C} : (\mathtt{B}, \mathtt{L}, \mathtt{N}) \leftarrow s_C(x)$ These are functions of x, L refers to each token in the sequence
$\Delta : (\mathtt{B}, \mathtt{L}, \mathtt{D}) \leftarrow \tau_\Delta$(Parameter$+s_\Delta(x)$)
$\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathtt{B}, \mathtt{L}, \mathtt{D}, \mathtt{N}) \leftarrow$ discretize$(\Delta, \boldsymbol{A}, \boldsymbol{B})$
$y \leftarrow \mathrm{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
return y

Selective State Space (Algorithm)

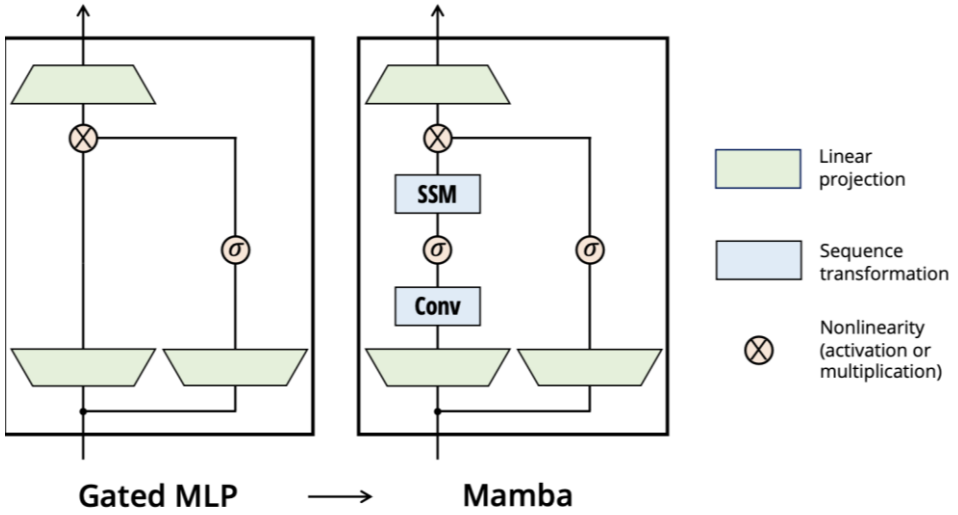
- The main difference is simply making several parameters functions of the input
  - Δ, B, C
- Algorithm S6 (Mamba) is time-varying
- Algorithm S4 is time-invariant
- **Mamba relaxes the idea of time transition between input sequences are independent**

Punchline

- Mamba moves S4 a little bit closer to LSTM models by introducing time-dependant transition from time step to time step in processing input sequence.
- It still preserve the quality of S4 in processing input sequence in one pass like transformers

Mamba Architecture:
A Simplified SSM Architecture

- Right figure shows Mamba Architecture
- Mamba simplify this architecture by combining MLP and linear attention components into one block
- It also has gating mechanism to keep or modify passed info through the network
- SSM block here is the previous selective state architecture shown in slide 17:



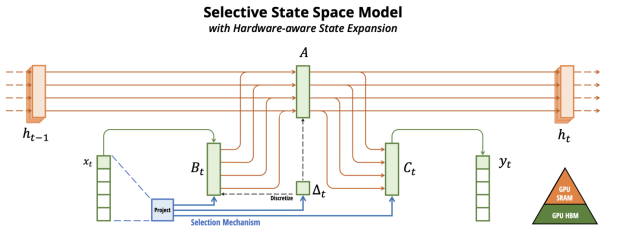


Figure 1: (**Overview**.) Structured SSMs independently map each channel (e.g. $D = 5$) of an input x to output y through a higher dimensional latent state h (e.g. $N = 4$). Prior SSMs avoid materializing this large effective state (DN, times batch size B and sequence length L) through clever alternate computation paths requiring time-invariance: the $(\Delta, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C})$ parameters are constant across time. Our selection mechanism adds back input-dependent dynamics, which also requires a careful hardware-aware algorithm to only materialize the expanded states in more efficient levels of the GPU memory hierarchy.

Empirical Evaluation

- **Selective Copying task**: synthetic tasks for sequence modeling, designed to test the memorization abilities of recurrent models

Model	Arch.	Layer	Acc.
S4	No gate	S4	18.3
-	No gate	S6	**97.0**
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	**99.7**
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	**99.8**

Table 1: (**Selective Copying**.) Accuracy for combinations of architectures and inner sequence layers.

Empirical Evaluation

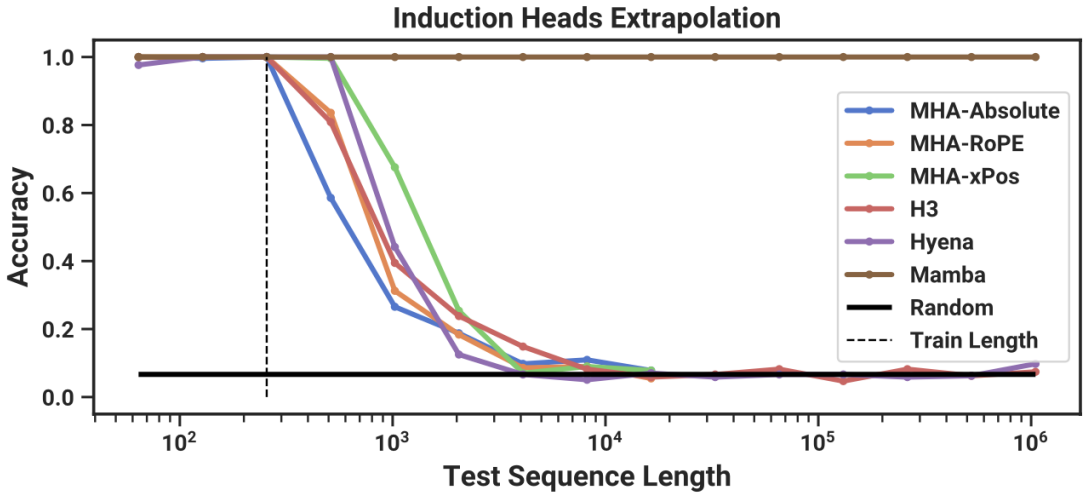


Table 2: (**Induction Heads**.) Models are trained on sequence length $2^8 = 256$, and tested on increasing sequence lengths of $2^6 = 64$ up to $2^{20} = 1048576$. Full numbers in Table 11.

- **Induction Heads**: requires models to perform associative recall and copy
- For example, if the model has seen a bigram such as “Harry Potter”, then the next time “Harry” appears in the same sequence, the model should be able to predict “Potter” by copying from history.
- This experiment shows the strength of Mamba for longer sequence length

Down Stream Evaluation (DNA Modeling)

Table 3: (**Zero-shot Evaluations**.) Best results for each size in bold. We compare against open source LMs with various tokenizers, trained for up to 300B tokens. Pile refers to the validation split, comparing only against models trained on the same dataset and tokenizer (GPT-NeoX-20B). For each model size, Mamba is best-in-class on every single evaluation result, and generally matches baselines at twice the model size.

Model	Token.	Pile ppl ↓	LAMBADA ppl ↓	LAMBADA acc ↑	HellaSwag acc ↑	PIQA acc ↑	Arc-E acc ↑	Arc-C acc ↑	WinoGrande acc ↑	Average acc ↑
Hybrid H3-130M	GPT2	—	89.48	25.77	31.7	64.2	44.4	24.2	50.6	40.1
Pythia-160M	NeoX	29.64	38.10	33.0	30.2	61.4	43.2	24.1	**51.9**	40.6
Mamba-130M	NeoX	**10.56**	**16.07**	**44.3**	**35.3**	**64.5**	**48.0**	**24.3**	**51.9**	**44.7**
Hybrid H3-360M	GPT2	—	12.58	48.0	41.5	68.1	51.4	24.7	54.1	48.0
Pythia-410M	NeoX	9.95	10.84	51.4	40.6	66.9	52.1	24.6	53.8	48.2
Mamba-370M	NeoX	**8.28**	**8.14**	**55.6**	**46.5**	**69.5**	**55.1**	**28.0**	**55.3**	**50.0**
Pythia-1B	NeoX	7.82	7.92	56.1	47.2	70.7	57.0	27.1	53.5	51.9
Mamba-790M	NeoX	**7.33**	**6.02**	**62.7**	**55.1**	**72.1**	**61.2**	**29.5**	**56.1**	**57.1**
GPT-Neo 1.3B	GPT2	—	7.50	57.2	48.9	71.1	56.2	25.9	54.9	52.4
Hybrid H3-1.3B	GPT2	—	11.25	49.6	52.6	71.3	59.2	28.1	56.9	53.0
OPT-1.3B	OPT	—	6.64	58.0	53.7	72.4	56.7	29.6	59.5	55.0
Pythia-1.4B	NeoX	7.51	6.08	61.7	52.1	71.0	60.5	28.5	57.2	55.2
RWKV-1.5B	NeoX	7.70	7.04	56.4	52.5	72.4	60.5	29.4	54.6	54.3
Mamba-1.4B	NeoX	**6.80**	**5.04**	**64.9**	**59.1**	**74.2**	**65.5**	**32.8**	**61.5**	**59.7**
GPT-Neo 2.7B	GPT2	—	5.63	62.2	55.8	72.1	61.1	30.2	57.6	56.5
Hybrid H3-2.7B	GPT2	—	7.92	55.7	59.7	73.3	65.6	32.3	61.4	58.0
OPT-2.7B	OPT	—	5.12	63.6	60.6	74.8	60.8	31.3	61.0	58.7
Pythia-2.8B	NeoX	6.73	5.04	64.7	59.3	74.0	64.1	32.9	59.7	59.1
RWKV-3B	NeoX	7.00	5.24	63.9	59.6	73.7	67.8	33.1	59.6	59.6
Mamba-2.8B	NeoX	**6.22**	**4.23**	**69.2**	**66.1**	**75.2**	**69.7**	**36.3**	**63.5**	**63.3**
GPT-J-6B	GPT2	–	4.10	68.3	66.3	75.4	67.0	36.6	64.1	63.0
OPT-6.7B	OPT	–	4.25	67.7	67.2	76.3	65.6	34.9	65.5	62.9
Pythia-6.9B	NeoX	6.51	4.45	67.1	64.0	75.2	67.3	35.5	61.3	61.7
RWKV-7.4B	NeoX	6.31	4.38	67.2	65.5	76.1	67.8	37.5	61.0	62.5

Summary

- Introduction to State Space Models
  - Weakness of Transformers
  - RNN Representation
  - CNN Representation
  - HiPPO Matrix
  - S4 models
- Mamba
  - Benefits of Mamba
  - Selective State Space block
  - Comparison with S4
  - Connection with LSTM